ST. VINCENT PALLOTTI
COLLEGE OF ENGINEERING & TECHNOLOGY, NAGPUR
AN AUTONOMOUS INSTITUTION

Approved by the AICTE (F.No.06/07/MS/ENGG/2004/029), DTE(2NGC/2004/Mantralaya/7) &
Affiliated to Nagpur University (NAVI/MVI/04/694/S/158)
(Managed by The Nagpur Pallottine Society)
Accredited by NAAC with 'A' Grade Post Box No. 128, G.P.O. Nagpur 440001

DEPARTMENT OF COMPUTER SCIENCE & BUSINESS SYSTEMS

Assignment 2

Course: Design and Analysis of Algorithms
Course Code: 24CB501T
Semester: 5th
Year: 3rd

UID: 23016027
Student Name: Shravan Vijay Sapate
Faculty Name: Prof. Pradnya S. Moon

Title: Case Study – FRIEND SUGGESTIONS USING GRAPH CLUSTERING

Pseudo Code

```
pgsql

Input: n (users), m (friendships)
Create graph G with n nodes

FOR each friendship (u, v)
    Add v to G[u]
    Add u to G[v]
END FOR

visited[n] = false
clusters = []

FOR each user i in [0..n-1]
    IF not visited[i]
        cluster = []
        DFS(i, cluster, visited, G)
        Add cluster to clusters
    END IF
END FOR

FOR each cluster in clusters
    FOR each user in cluster
        suggestions = []
        FOR each member in cluster
            IF member ≠ user AND member NOT in G[user]
                Add member to suggestions
            END IF
        END FOR
        Print "User", user, "→ Suggested:", suggestions
    END FOR
END FOR
```

DFS(node, cluster, visited, G):

```pgsql
visited[node] = true
Add node to cluster
FOR each neighbor in G[node]
    IF not visited[neighbor]
        DFS(neighbor, cluster, visited, G)
    END IF
END FOR
```

Example Input:

n = 5, m = 3 Edges: 0–1

1–2

3–4

Step 1: **Graph**

User	**Friends**
0	1
1	0, 2
2	1
3	4
4	3

Step 2:

Step	Node	Cluster Formed	Visited
1	0	[0,1,2]	✓✓✓□□
2	3	[3,4]	✓✓✓✓✓

Clusters → [0,1,2], [3,4]

Step 3:

Friend Suggestions

User	Direct Friends	Suggestions
0	1	2
1	0,2	–
2	1	0
3	4	–
4	3	–

Final Output:

User 0 → Suggested: 2

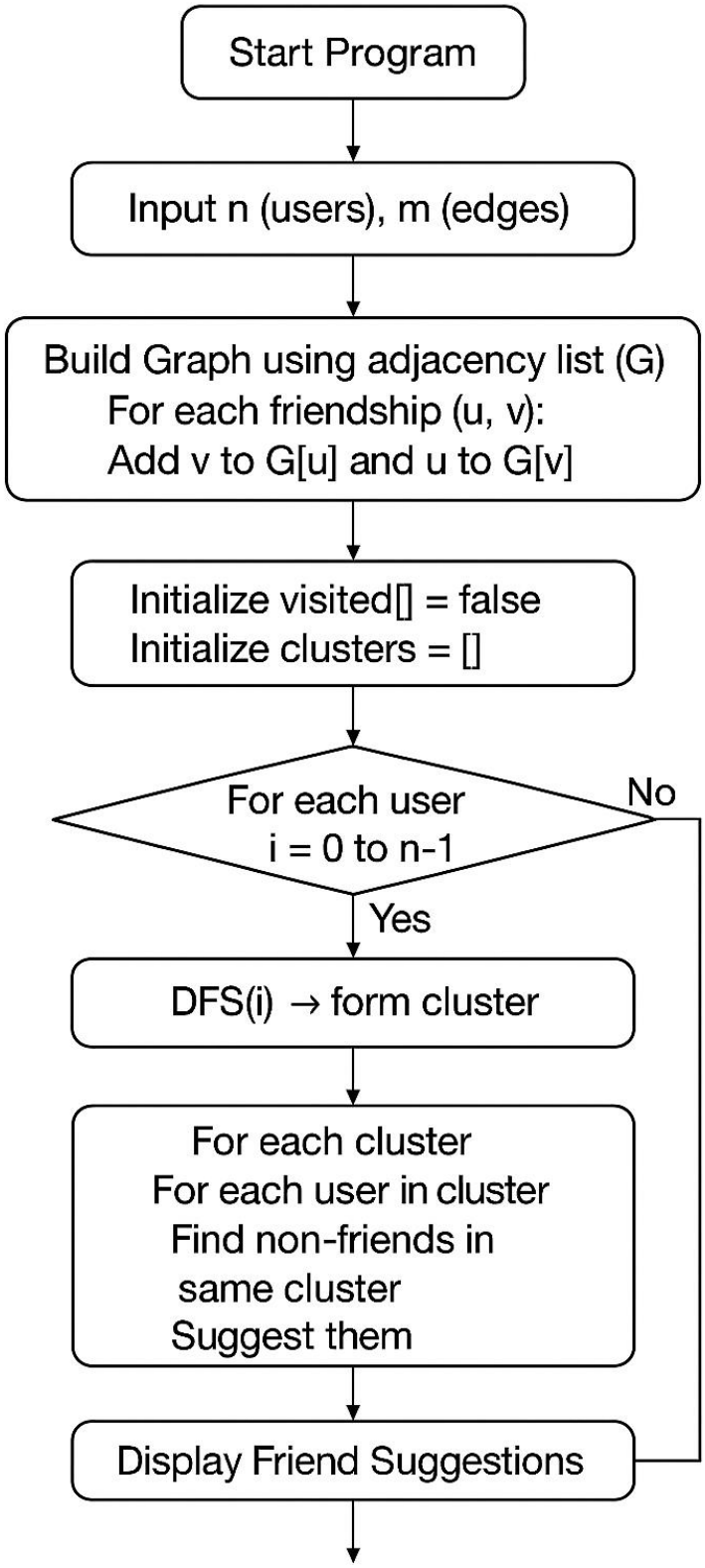
User 1 → No new suggestions

User 2 → Suggested: 0

User 3 → No new suggestions

User 4 → No new suggestions

Flow Chart:


Start Program
Input n (users), m (edges)
Build Graph using adjacency list (G)
For each friendship (u, v):
Add v to G[u] and u to G[v]
Initialize visited[] = false
Initialize clusters = []
For each user
i = 0 to n-1
No
Yes
DFS(i) → form cluster
For each cluster
For each user in cluster
Find non-friends in
same cluster
Suggest them
Display Friend Suggestions

Variables

- n, m → users & friendships
- G[] → adjacency list
- visited[] → DFS visit tracker
- clusters → list of user groups
- suggestions → recommended friends

Behavior

- Build graph → DFS to form clusters
- For each user in a cluster → suggest non-direct friends

For Each Edge

Add edge (u, v)

Add edge (v, u)

Stopping Condition

All users visited (DFS complete)

Output

Friend suggestions for each user in same cluster
e.g.
User 0 → Suggested: 2

Complexities

- **Time:** O(n + m) (sparse) or O(n^2) (dense)
- **Space:** O(n + m)